EDGE AI WITH THE RASPBERRY PI - DAY 02

Remote Access Like a Pro: Files, Desktop and More Than One Network

Work comfortably from any laptop: nicknames and keys for SSH, moving files both ways, the Pi desktop in a window, a second Wi-Fi network, and reaching a Pi across networks.

Time needed	2 hours, in six parts (A to F). Parts B to D are the core; E and F matter when you use more than one place or more than one Pi.
Boards covered	**Raspberry Pi 4** and **Raspberry Pi 5**. Everything works on both; the few differences are marked.
You need	A Pi that you can already log into over SSH (Day 1), your laptop (Windows steps are given; Mac and Linux differences are noted), and the connection card from Day 1.
Outcome	You connect with a short command, without typing a password; copy files in both directions; open the Pi desktop in a window; add a second Wi-Fi network to a Pi; and know your options for reaching a Pi in another place.
Previous / next	Day 01: meet the Raspberry Pi and bring it to life. Day 03: the Linux command line.

By the end of today you will be able to

- ☐ Give each Pi a nickname so that `ssh lab` is all you type.
- ☐ Log in with an SSH key instead of a password, and explain why it is safer.
- ☐ Keep a long job running when your connection drops (tmux).
- ☐ Copy files and folders between your laptop and the Pi in both directions, with `scp` and with an SFTP program.
- ☐ Keep code and notes synchronised between several computers with git.
- ☐ Open the Pi's graphical desktop on your laptop with VNC and know when SSH is enough.
- ☐ Add a second Wi-Fi network to a Pi and find a Pi's address when `.local` fails.
- ☐ Choose a safe way to reach a Pi that is in another network, and apply basic security habits.

Your 2-hour plan

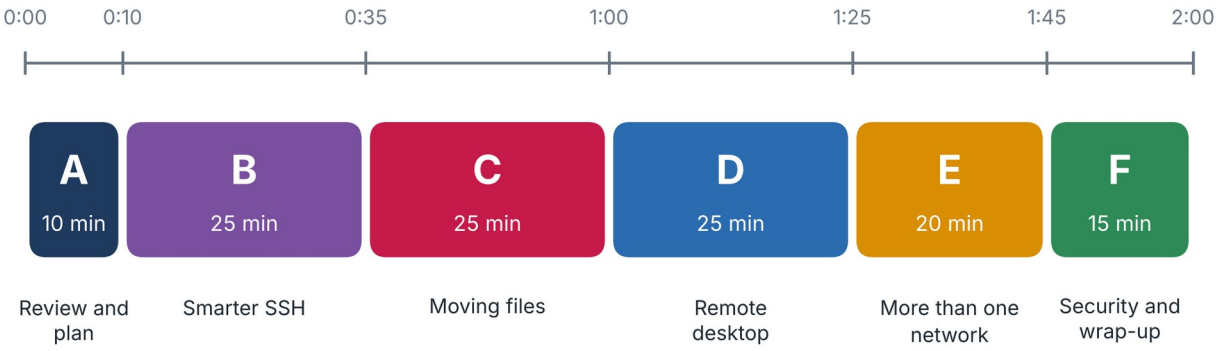


Figure 1. The two hours at a glance.

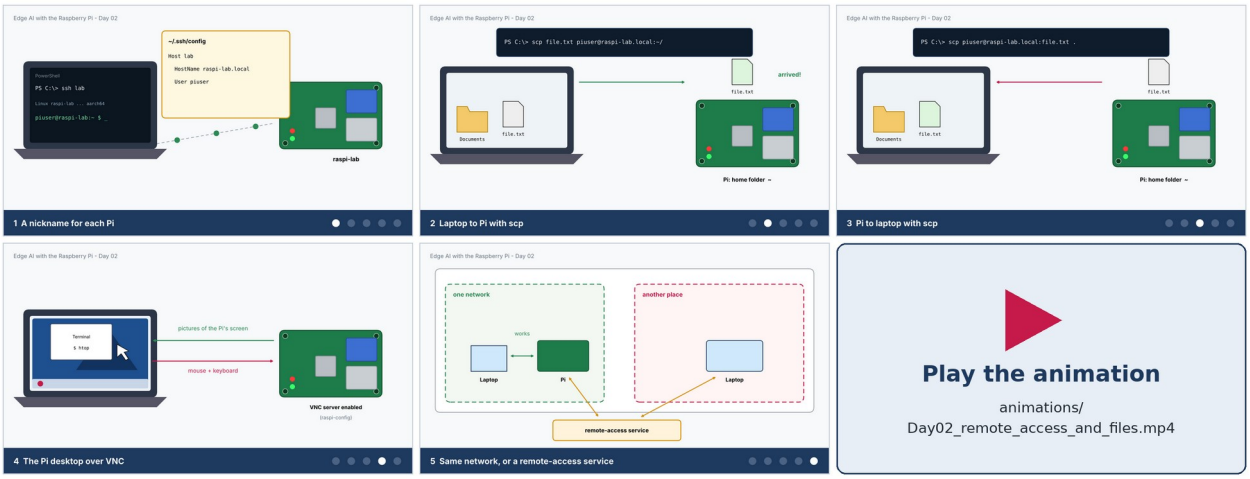


Figure 2. Key frames of the narrated animation 'Day02_remote_access_and_files.mp4' (about 65 seconds, English voice-over and subtitles): nickname, scp to the Pi, scp from the Pi, the VNC desktop, and the network picture. Play the video for the full story.

Part A - Review and plan (10 min)

A1. Where we are

On Day 1 you wrote an operating system to a microSD card, booted the Pi and logged in with `ssh piuser@raspi4.local`. That works, but it has rough edges: a long command, a password every time, no way to move files, no desktop, and it only works while the laptop and the Pi share one network. Today we remove each rough edge.

Rough edge on Day 1	Today's remedy	Part
Long command to type	A nickname in the SSH configuration file	B
Password every time	An SSH key pair	B
Job dies when the connection drops	tmux (a session that survives)	B
No way to move files	`scp`, an SFTP program, and git	C
Only a command line	VNC: the whole desktop in a window	D
One Wi-Fi network only	More than one Wi-Fi profile on the Pi	E
Laptop and Pi on different networks	Remote-access options, and what to avoid	F

A2. A naming plan for several Pis

If you have more than one Pi (or a class has many), names matter. Use a pattern that tells you where the Pi lives, and keep the user name the same everywhere:

Item	Suggestion	Example
Host name	`raspi-` plus the place or the owner	`raspi-lab`, `raspi-home`, `raspi-team3`
User name	the same on every Pi	`piuser`
SSH nickname	short, easy to type	`lab`, `home`
Connection card	one per Pi (Day 1, Appendix A)	board, host name, IP, Wi-Fi, notes

> **TIP Renaming a Pi**
>
> If both of your Pis are still called `raspi4`, rename one: on the Pi run `sudo raspi-config`, choose System Options, then Hostname, type the new name and reboot. (The old name stops working and the new `.local` name takes over.)

Part B - Smarter SSH (25 min)

B1. A nickname for each Pi: the SSH configuration file

The SSH client reads a small text file where you can describe each machine once. After that, `ssh`, `scp` and many other tools accept the nickname instead of the long form.

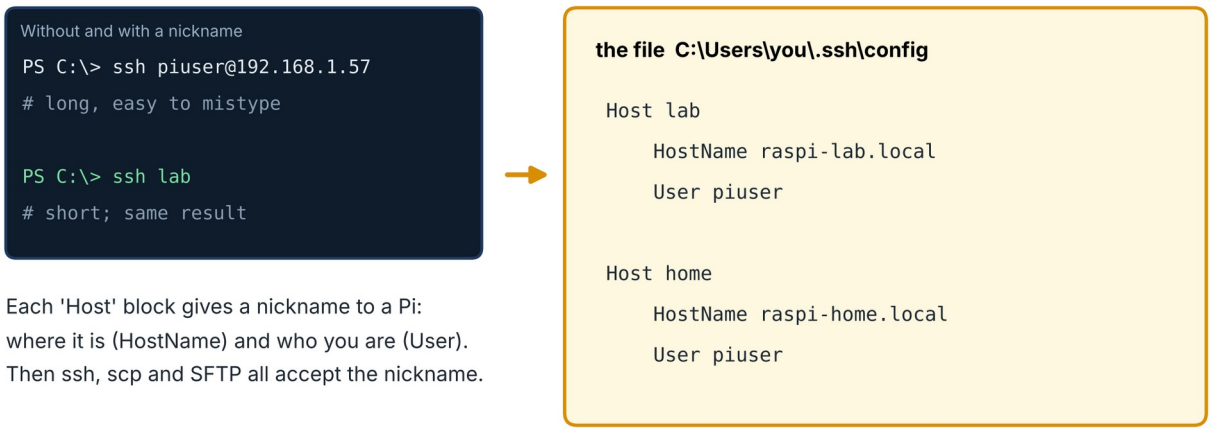


Figure 3. The configuration file turns a nickname into a full destination.

1 Create the file

In PowerShell, create the folder and an empty file named exactly `config` (no `.txt` extension), then open it in Notepad:

```
New-Item -ItemType Directory -Force $env:USERPROFILE\.ssh | Out-Null
New-Item -ItemType File -Force $env:USERPROFILE\.ssh\config | Out-Null
notepad $env:USERPROFILE\.ssh\config
```

2 Describe your Pis

Type one block per Pi (the indented lines start with spaces), then save and close Notepad. Use your own host names and user name.

```
Host lab
    HostName raspi-lab.local
    User piuser

Host home
    HostName raspi-home.local
    User piuser

# options for every host:
Host *
    ServerAliveInterval 60
```

3 Test it

Connect using only the nickname. The first time for each Pi you are asked to trust it (`yes`) and for the password.

```
ssh lab
```

Line	Meaning
`Host lab`	starts a block; `lab` is the nickname you will type

Line	Meaning
`HostName ...`	the real destination: a `.local` name or an IP address (use the IP if `.local` is unreliable on your network)
`User piuser`	the user to log in as, so you can leave out `piuser@`
`Host */ServerAliveInterval 60`	applies to all hosts; sends a signal every 60 seconds so idle connections are not dropped by routers

YOUR SCREENSHOT GOES HERE

S1 - PowerShell: connecting with a nickname

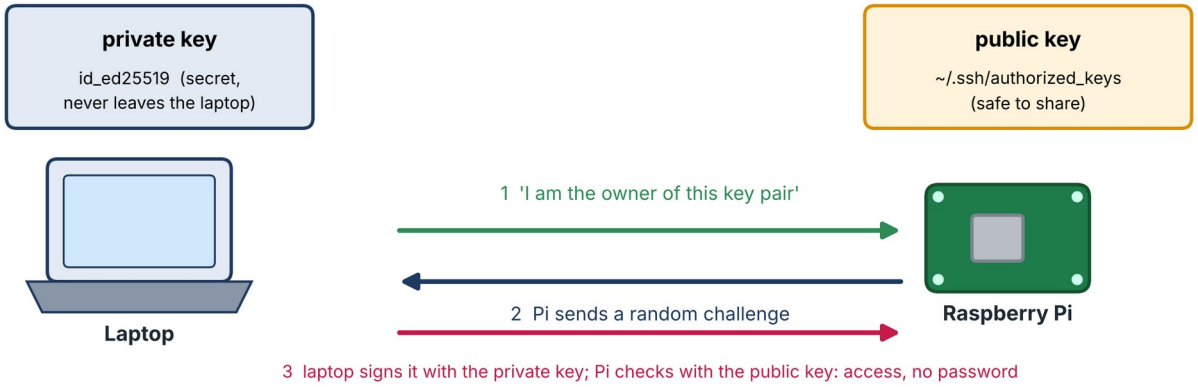
Capture the window after ssh lab shows the Pi prompt.
Save as S1_ssh_alias.png in the folder my_screenshots/day02/

Figure 4. Connecting with a nickname (capture slot: add your own screenshot).

> **NOTE Mac and Linux**
> The file is `~/.ssh/config` and the contents are identical. Create it with `mkdir -p ~/.ssh` and `nano ~/.ssh/config`.

B2. SSH keys: log in without a password

A password can be guessed, typed in the wrong place or stolen. An **SSH key pair** is stronger. You create two linked files on your laptop: a **private key** (secret, never leaves the laptop) and a **public key** (harmless to share). You place the public key on the Pi. Later the Pi challenges your laptop to prove that it holds the private key, and lets you in. The secret is never sent over the network.



Keys prove who you are without sending a secret over the network.

Figure 5. How key-based login works.

1 Create a key pair on the laptop

Run the command below. Press Enter to accept the default file name. When asked for a passphrase you may press Enter for none, or type one: a passphrase protects the key if someone copies your laptop's files. The `-C` text is just a label.

```
ssh-keygen -t ed25519 -C "office-laptop"
# creates  %USERPROFILE%\.ssh\id_ed25519       (private: keep secret)
#          %USERPROFILE%\.ssh\id_ed25519.pub   (public: safe to share)
```

2 **Copy the public key to the Pi**

This command (in PowerShell) sends your public key to the Pi and appends it to the list of keys the Pi accepts. You are asked for the password one last time.

```
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh lab "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
# (on Mac or Linux:  ssh-copy-id lab )
```

3 **Test**

Log in again. It should now open the Pi's prompt immediately (or ask for your key passphrase, not the Pi password).

```
ssh lab
```

CAREFUL Rules for keys

- **Never copy or send the private key** (the file without `.pub`) to anyone or to another computer.
- **One key per laptop.** If you use two laptops (for example office and home), create a key on each and add **both** public keys to each Pi (repeat step 2 from each laptop).
- If you re-write a Pi's card, its `authorized_keys` is gone: repeat step 2.
- Keep password login enabled until you have tested the key. Turning it off is covered in Part G.

B3. tmux: sessions that survive a dropped connection

When you start a long job over SSH (for example a model download or training) and the connection drops, the job normally dies with your terminal. **tmux** keeps your terminal alive on the Pi so that you can leave and come back.



Figure 6. With tmux the job keeps running when the connection drops.

```
sudo apt install tmux        # once
tmux new -s work              # start a session named 'work'
# ... run your long command inside it ...
# press  Ctrl+b  then  d      # detach: leave it running
tmux ls                       # list sessions
tmux attach -t work           # come back later, from any laptop
exit                          # inside tmux: end the session
```

Keys (press Ctrl+b first)	Does
d	detach (leave the session running)
[	scroll back with the arrow keys; q to leave

Keys (press Ctrl+b first)	Does
`c`	new window inside the session
`n` / `p`	next / previous window

B4. Small habits that save time

Do	Result
Press the Up arrow	recalls previous commands
Press Tab while typing a name	completes file and folder names
`history`	lists past commands
Ctrl+C	stops the running command
`clear` (or Ctrl+L)	cleans the screen
`exit` (or Ctrl+D)	leaves the SSH session

Part C - Moving files (25 min)

C1. scp: copy over SSH

scp (secure copy) copies files through the same encrypted channel as SSH, so it needs nothing extra on the Pi. The pattern is always the same: **scp SOURCE DESTINATION**. Anything written as `name:path` is on the Pi; a plain path is on the computer you are typing on.

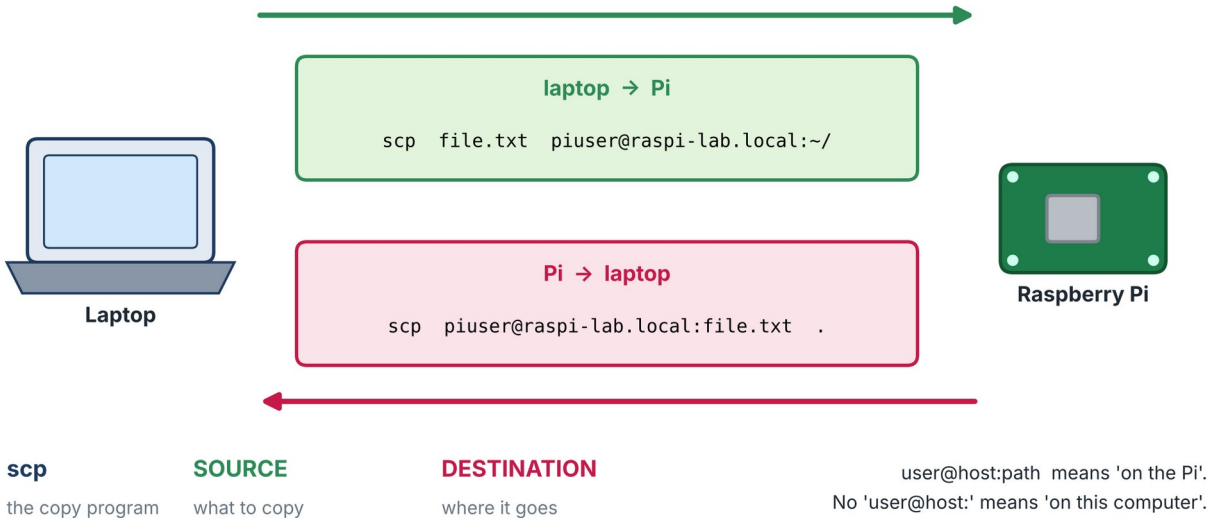


Figure 7. scp copies from a source to a destination. The side with 'user@host:' or 'nickname:' is the Pi.

Practice (10 minutes)

1 **Make a test file on the laptop**

Open PowerShell in a folder you can write to (for example `cd $HOME\Documents`):

```
"Hello from my laptop" | Out-File test.txt -Encoding ascii
```

2 **Send it to the Pi**

The colon after the nickname is required. `~/` means your home folder on the Pi.

```
scp test.txt lab:~/
```

3 Check it on the Pi

```
ssh lab
ls -l
cat test.txt
cp test.txt test_2.txt      # make a second copy on the Pi
exit
```

4 Fetch a file from the Pi

The Pi's file comes first. The final dot means 'the current folder on this computer'.

```
scp lab:test_2.txt .
```

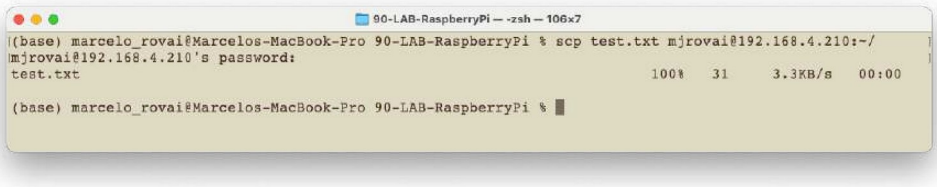


Figure 8. A scp transfer: the file name, 100%, size and speed are shown (example on a Mac, using an IP address instead of a nickname).

More scp

Task	Command	Notes
Copy a whole folder	`scp -r photos lab:~/images/`	`-r` = recursive. The destination folder must already exist.
Create a folder first	`ssh lab "mkdir -p ~/images"`	scp never creates folders for you.
Several files	`scp a.jpg b.jpg lab:~/images/`	or `scp *.jpg lab:~/images/` (the laptop expands the `*`).
From the Pi, with a wildcard	`scp "lab:~/images/*.jpg" .`	Quote it so the Pi, not your laptop, expands the `*`.
Another folder on the Pi	`scp test.txt lab:~/projects/day02/`	Use `~/` for your home folder.
Spaces in names	`scp "my file.txt" lab:~/`	Quote names with spaces, or avoid spaces.
A different port	`scp -P 2222 test.txt lab:~/`	Capital `-P` for scp (SSH uses lowercase `-p`).

TIP Windows paths

Open PowerShell in the folder that holds the file (or use `cd`) and use short relative names. A full path such as `C:\Users\you\file.txt` contains a colon, which can confuse `scp` about what is local and what is on the Pi.

C2. An SFTP program with two panes

If you prefer to drag and drop, use an **SFTP** program: it speaks the same secure protocol as `scp` but shows two panes, your laptop on one side and the Pi on the other. Popular free choices on Windows are **FileZilla** and **WinSCP**. Download either from the project's official site (see the references) and decline any extra software offered in the installer.

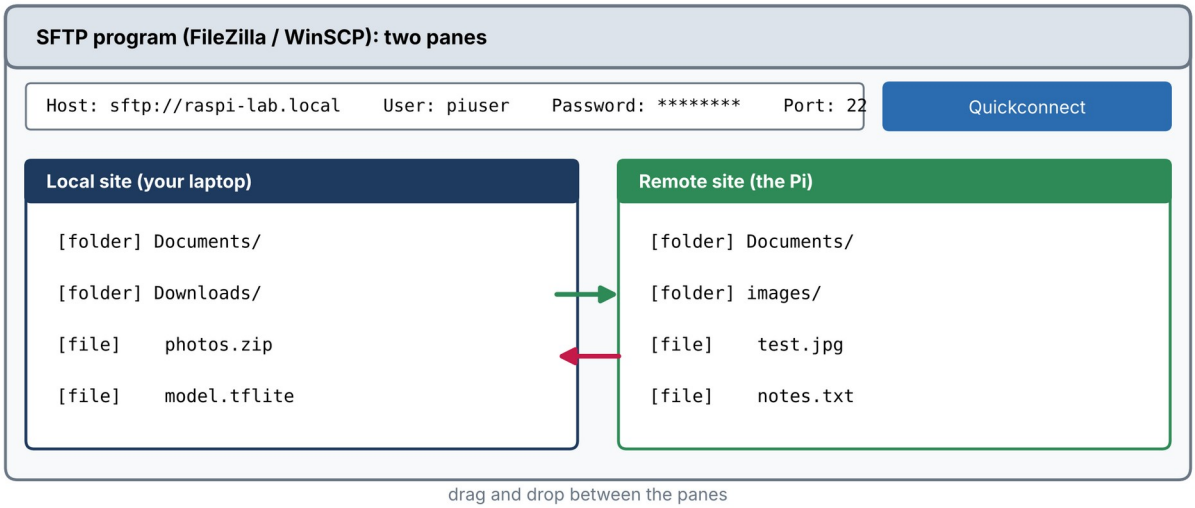


Figure 9. An SFTP program: connect once, then drag files between the two panes.

Field	Value
Host	`sftp://raspi-lab.local` (or the Pi's IP address)
Protocol (WinSCP)	SFTP
User name	`piuser`
Password	your Pi password (or choose a key file in the program's key settings)
Port	22 (leave blank for the default)

Press **Quickconnect** (FileZilla) or **Login** (WinSCP). Accept the unknown-host message the first time. Save the connection in the program's site manager so that you do not retype it.

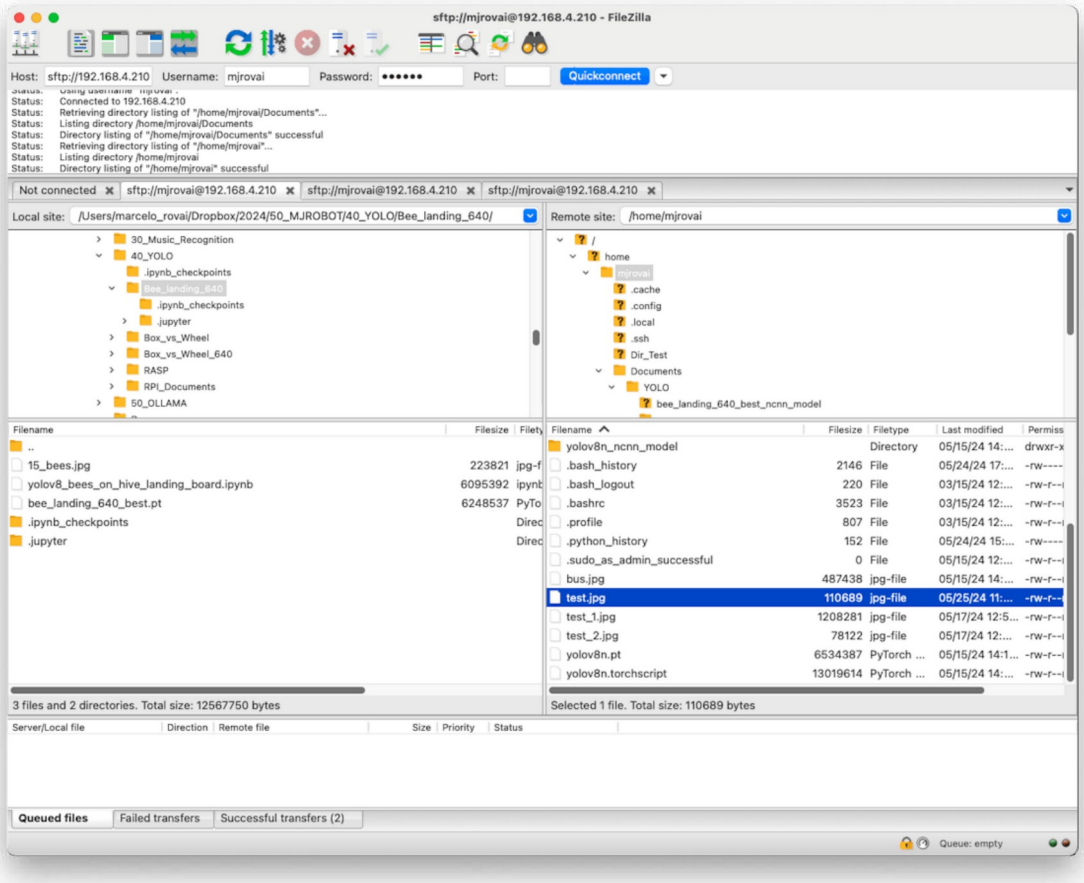


Figure 10. FileZilla connected to a Pi: local files on one side, the Pi's files on the other (example on a Mac).

YOUR SCREENSHOT GOES HERE

S2 - your SFTP program connected to your Pi

Capture FileZilla or WinSCP with both panes visible (hide the password).
Save as S2_sftp.png

Figure 11. Your SFTP program connected to your Pi (hide the password). (capture slot: add your own screenshot)

C3. Keeping several computers in sync with git

With two laptops and two Pis, copying files by hand gets confusing: which copy is the newest? A better habit is to keep your **code and notes in an online repository** (for example on GitHub) and let every computer synchronise with it.

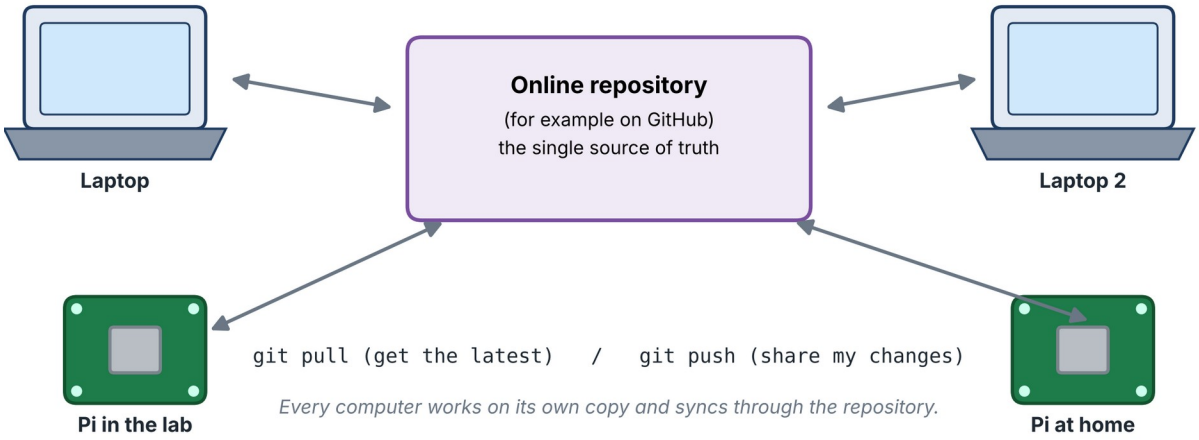


Figure 12. Every computer keeps its own copy; the online repository is the single source of truth.

```
sudo apt install git                              # once, on the Pi
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
git clone https://github.com/YOUR-NAME/YOUR-REPO.git   # first time: get a copy
cd YOUR-REPO
git pull                                           # get the latest changes
# ... edit files ...
git add .                                          # choose what to share
git commit -m "Describe the change"
git push                                           # share it
```

NOTE Signing in to GitHub from the Pi

GitHub no longer accepts your account password for `git push` over HTTPS. Create a **personal access token** (or an SSH key for the Pi) in your GitHub account settings and give the token when git asks for a password. Keep the token private, like a password.

CAREFUL What not to put in git

Large files (model files of hundreds of megabytes, datasets, videos) do not belong in an ordinary repository. Keep them out with a `.gitignore` file and move them with `scp` or an SFTP program instead.

C4. Which tool when?

Tool	Best for	Remember
`scp`	a few files or a folder, scripts, one-off copies	fast to type; needs the nickname or address
SFTP program	browsing the Pi's folders, many files, drag and drop	good for beginners; save the connection
git	code and notes shared by several computers	history of changes; not for big files
USB drive	very large data when the network is slow	safe to remove only after `sync` or shutdown

Part D - The Pi desktop in a window (25 min)

D1. SSH or desktop?

	SSH (command line)	VNC (desktop)
What you see	text only	the whole graphical desktop, windows and mouse
Load on the Pi	very small	noticeably more (the desktop must run and be drawn)
Network needed	very little	much more bandwidth
Best for	installing, running, editing, automation	browsers, image viewers, graphical tools, learning
Needs the full OS image	no (works with Lite too)	yes: the Pi must run a desktop

In this course you will do most work over SSH and use the desktop when a graphical window helps (for example to see images, a camera preview or a plot).

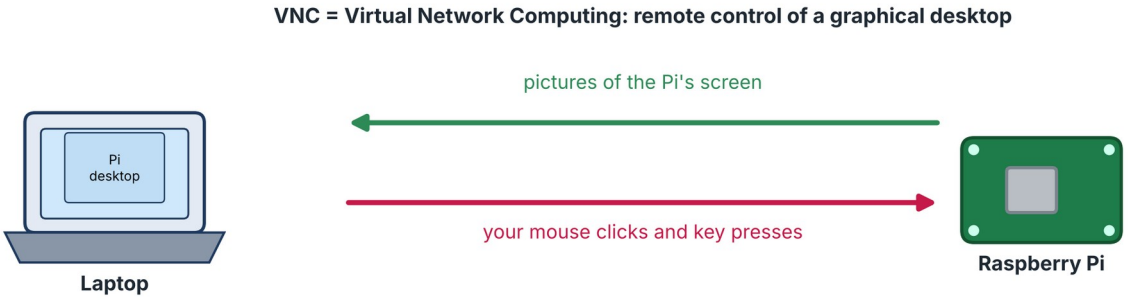


Figure 13. VNC sends pictures of the Pi's screen to your laptop and your mouse and keyboard back.

D2. Turn on the VNC server on the Pi

1 **Open the configuration tool**

Over SSH, run:

```
sudo raspi-config
```

2 **Enable VNC**

Use the arrow keys: **Interface Options**, then **VNC**, then **Yes**. Finish and accept a reboot if offered. (On the desktop you can do the same in Raspberry Pi Configuration, Interfaces tab.)

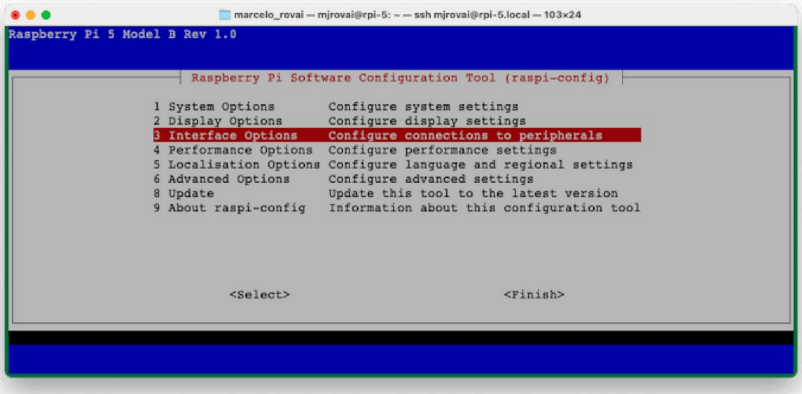


Figure 14. raspi-config: the top menu. Choose 'Interface Options' (example screen).

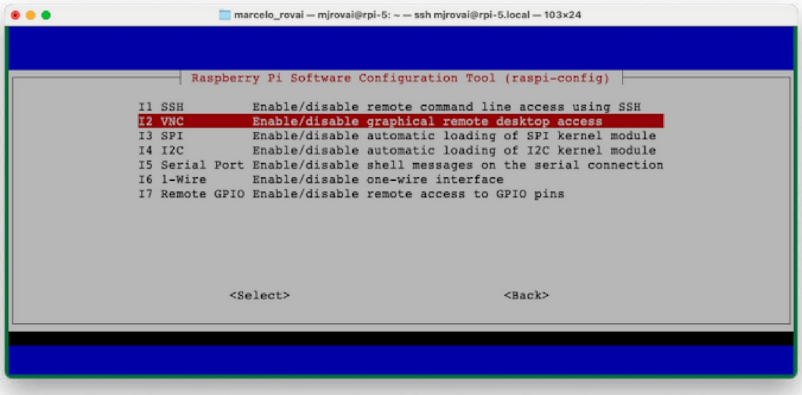


Figure 15. raspi-config: the interface options, including VNC (example screen).

3 **Know the address**

`hostname -I` shows the IP address; the host name also works: `raspi-lab.local`.

> **NOTE** **Black screen or tiny window when no monitor is attached?**
>
> A Pi with no monitor has no 'real' screen, so the desktop may be black or very small. In `sudo raspi-config` open **Display Options** and set a resolution (for example 1920x1080). Which VNC program runs on the Pi depends on the OS release: older releases use RealVNC Server; recent releases that use the Wayland desktop can use a different server (wayvnc). If your viewer refuses to connect or shows nothing, `raspi-config` -> Advanced Options lets you switch between the Wayland and X11 desktops; X11 with RealVNC is the most widely documented combination.

D3. Connect from the laptop

1 **Install a VNC viewer**

Install **RealVNC Viewer** (free; see the references) or another VNC viewer on your laptop.

2 **Connect**

Type the Pi's address (`raspi-lab.local` or its IP) in the address bar and press Enter. Accept the identity (fingerprint) message the first time.

3 **Sign in with your Pi account**

Enter your **Pi** user name and password (not an account of the VNC company). The Pi's desktop appears in a window.

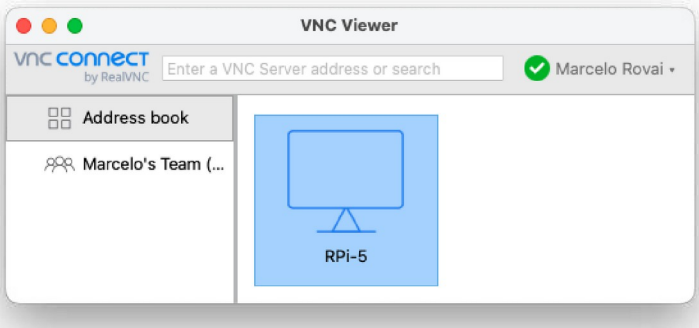


Figure 16. The viewer's window with a saved Pi (example on a Mac).

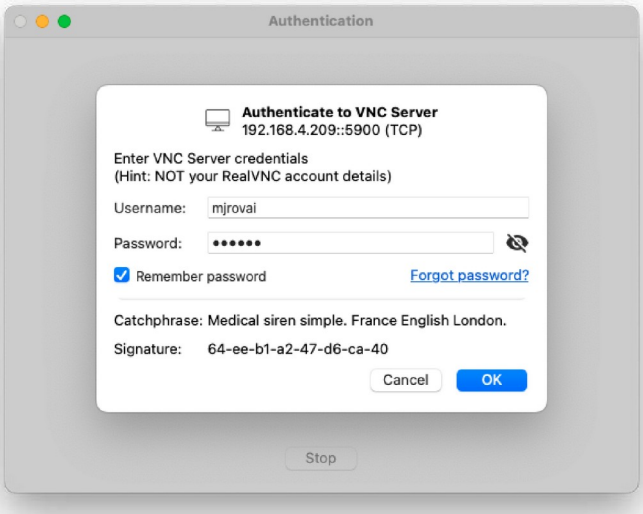


Figure 17. The sign-in window: use the Pi's own user name and password.

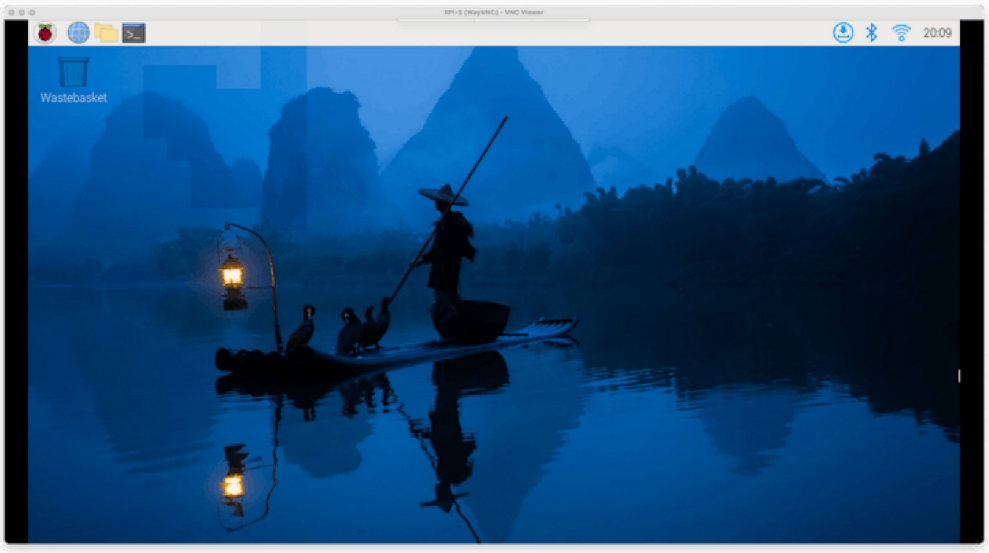


Figure 18. The Pi desktop appears in a window on your laptop.

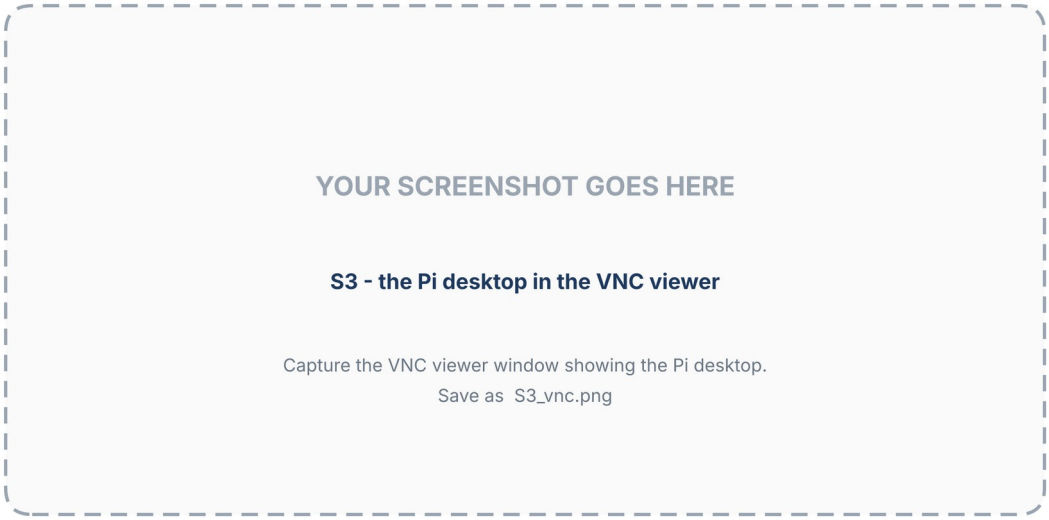


Figure 19. The Pi desktop in your VNC viewer. (capture slot: add your own screenshot)

D4. Getting a comfortable desktop

- **Resolution:** in the Pi's desktop open Preferences, Screen Configuration (or Display Options in `raspi-config`) and choose a size that fits your laptop screen.
- **Speed:** over a slow network reduce the picture quality or colour depth in the viewer's options, and remove the desktop wallpaper.
- **Copy and paste** between laptop and Pi usually works in the viewer; if not, check the clipboard option.
- **Close it when finished.** A desktop session uses memory that your models will need later.
- **Security:** never open VNC's port (5900) to the Internet. Use it on your own network or inside a private connection (Part F).

Part E - More than one network (20 min)

E1. A Pi can remember several networks

Imager stored one Wi-Fi network for you. A Pi, like a phone, can remember many and join whichever it finds. That lets the same Pi work at the lab and at home without re-writing the card.

E2. Add a Wi-Fi network (current Raspberry Pi OS)

Recent Raspberry Pi OS releases manage networks with **NetworkManager**, controlled from the command line with `nmcli`. First look at what is there:

```
nmcli connection show          # saved connections
nmcli device status               # network devices; wlan0 is the Wi-Fi
nmcli device wifi list            # Wi-Fi networks in range now
```

If the new network is **in range now**, connect to it directly:

```
sudo nmcli device wifi connect "HomeWiFi" password "the-password"
```

If the network is **not in range** (for example you are preparing the Pi at the lab for use at home), create a saved profile instead:

```
sudo nmcli connection add type wifi con-name home ifname wlan0 ssid "HomeWiFi" \
     wifi-sec.key-mgmt wpa-psk wifi-sec.psk "the-password"
sudo nmcli connection modify home connection.autoconnect yes
connection.autoconnect-priority 10
```

Command	Does
`nmcli connection show`	lists saved networks and profiles
`... autoconnect-priority 10`	a higher number is preferred when several networks are in range
`sudo nmcli connection delete home`	forgets a profile
`sudo nmtui`	a menu-driven version: Edit a connection -> Add -> Wi-Fi

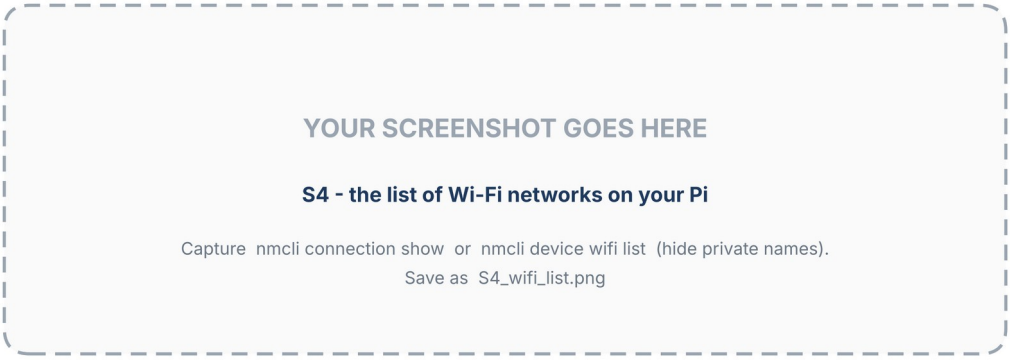


Figure 20. The saved Wi-Fi connections on your Pi (hide private names). (capture slot: add your own screenshot)

CAREFUL Do not cut your own branch

If you are connected to the Pi over Wi-Fi and you switch it to another network, your SSH session ends. Add profiles first, test later, and keep a monitor, keyboard or an Ethernet cable available as a way back in.

NOTE Older Raspberry Pi OS releases

Releases older than 'Bookworm' keep Wi-Fi settings in `/etc/wpa_supplicant/wpa_supplicant.conf` (one `network={ ssid="..." psk="..." }` block per network) or in the Wireless LAN option of `raspi-config`. Check your release with `cat /etc/os-release`.

E3. Finding the Pi when `.local` does not work

Some networks block the name announcements that make `raspi-lab.local` work. Then you need the Pi's IP address:

- **Router page:** log in to your router (its address is often `192.168.1.1` or `192.168.0.1`) and look at the list of connected devices for the host name.
- **From Windows:** `ping raspi-lab.local` shows an address if the name works; `arp -a` lists devices your laptop has talked to recently. Raspberry Pi devices usually have physical (MAC) addresses starting with `b8-27-eb`, `dc-a6-32` or `e4-5f-01` (newer boards use more prefixes).
- **With a screen:** plug a monitor into the Pi and run `hostname -I`.

Once you know it, put the IP address in the `HostName` line of your SSH configuration file.

E4. Keeping the address stable

The router hands out addresses (DHCP) and may give the Pi a new one after a reboot. Two ways to avoid surprises: use the `.local` name where it works, or ask the router to **always give the same address to this Pi** (called a *DHCP reservation* or *static lease*: you enter the Pi's MAC address in the router). Prefer this to setting a fixed address on the Pi itself, because the router stays in charge of the numbers.

E5. Two fallbacks

- **Phone hotspot:** put the Pi and the laptop on your phone's hotspot. This avoids guest-network and enterprise restrictions.
- **Ethernet cable straight between laptop and Pi:** after about a minute both sides give themselves link-local addresses (`169.254.x.x`) and `raspi-lab.local` often works. Useful when there is no Wi-Fi at all.

CAREFUL Office and campus networks

Many organisations only allow registered devices, or block device-to-device traffic. If the Pi will not join, ask the network administrator about a guest or IoT network or about registering the Pi's MAC address, and do not try to work around the rules.

Part F - Reaching a Pi in another network (20 min)

F1. Why it is hard

Your laptop at home and your Pi in the lab are on two private networks. Each router hides its devices behind one public address and refuses unsolicited connections from the Internet, and `.local` names never leave their own network. Something must connect the two sides.

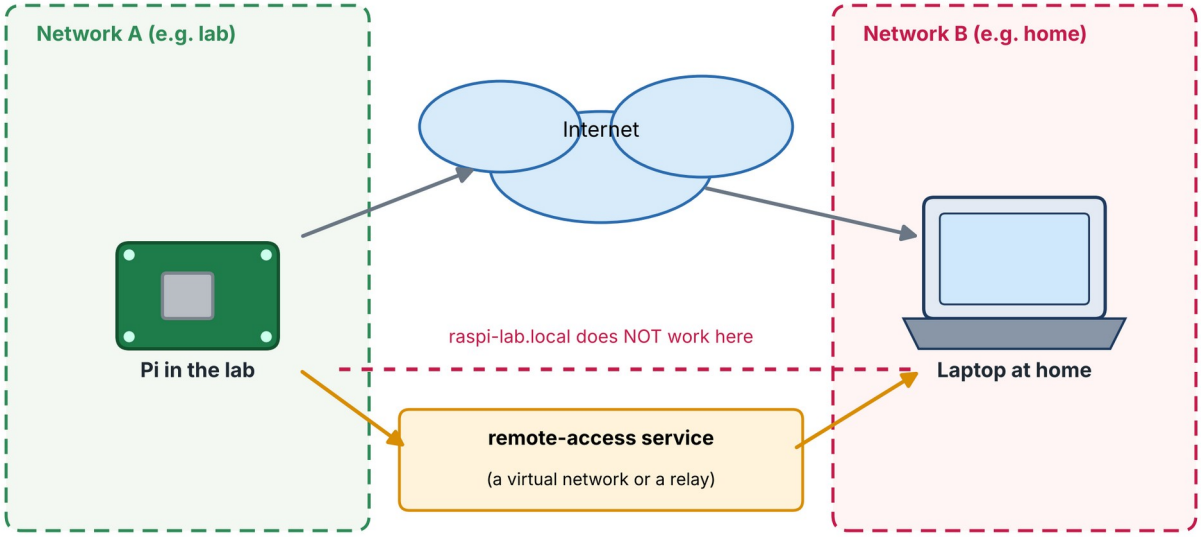


Figure 21. Two private networks cannot see each other directly. A remote-access service gives both sides a way to meet.

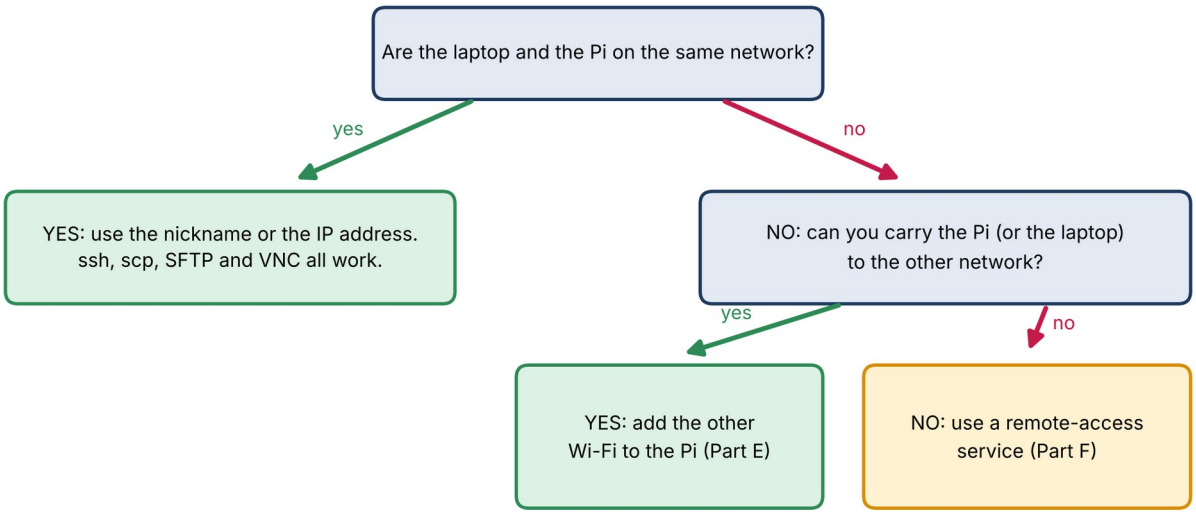


Figure 22. Choosing an approach.

F2. The options

Option	How it works	Good	Careful
Carry the Pi and add the other Wi-Fi (Part E)	the Pi joins whichever network it finds	simplest; nothing extra to install	you must physically move the Pi

Option	How it works	Good	Careful
A private-network service such as Tailscale	each device installs a small program and signs in with one account; devices then get private addresses and names that work from anywhere	works through home and office routers without changing them; every device stays private	software on your devices; needs a free account; check your organisation's policy first
Raspberry Pi Connect	the Pi maker's own service for a remote screen and a shell in your browser; it needs a recent OS release	from the maker; no router changes	follow the current official instructions; requires an account and Internet on the Pi
A VPN run by your organisation	once connected, your laptop behaves as if it were inside that network	no extra software on the Pi; approved by IT	only exists if your organisation provides it
Port forwarding on the router	opens SSH to the whole Internet	no extra software	**avoid**: automated attacks start within minutes; do not do this on a beginner setup

F3. Example: a private network with Tailscale

This example shows the idea. The vendor may change details, so check the current instructions on its website. Steps: create a free account; install the program on the laptop and sign in; install it on each Pi and sign in with the same account; the devices can then reach each other by the names shown in the service's device list.

```
# on the Pi (read the script first if you can; it is the vendor's installer):
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up       # prints a web address; open it and sign in
tailscale ip -4          # shows the Pi's private address, for example 100.x.y.z
```

CAREFUL Piping a script into the shell

`curl ... | sh` downloads a script and runs it immediately with your privileges. Only do it for a source you trust, and prefer the package-repository instructions on the vendor's page. Never run it on a work computer without permission.

CAREFUL Policy first

Installing remote-access software on a work laptop, or opening a path into a work network, can break your organisation's rules. Ask the IT department before you try Part F on any work device or network.

F4. A sensible plan for two Pis and two laptops

- Keep each Pi in **one place** and add the other place's Wi-Fi as a second profile (Part E) so you can also carry it.
- Give every Pi a nickname in the SSH configuration file on every laptop; use key login.
- Keep code and notes in git; move big files with scp.
- For access from outside, choose **one** approved method from Part F2; do not combine several.

Part G - Security habits (15 min)

Habit	Why and how
Change weak passwords	Use a long, unique password for the Pi user. Change it with `passwd`.
Keep the system updated	`sudo apt-get update && sudo apt upgrade` regularly (Day 1). Updates fix security problems.
Use keys; keep the private key private	Part B2. A key cannot be guessed over the network.
Never expose SSH or VNC to the Internet	Reach them only on your own network or through a private connection (Part F).
Know who is logged in	`who` shows current sessions; `last` shows recent logins.

Habit	Why and how
Keep secrets out of git and screenshots	Passwords, Wi-Fi keys and access tokens must not be committed or pasted into shared files.
Back up your work	git for code and notes; `scp` for important data. An SD card can fail.

Optional: turn password login off

After key login works from **every** computer you use, you can forbid passwords on the Pi so that guessing becomes impossible. Do this carefully: a mistake can lock you out of a Pi that has no screen.

```
sudo nano /etc/ssh/sshd_config
#   set the line:   PasswordAuthentication no
sudo systemctl restart ssh
# keep your current SSH window open; open a NEW window and confirm that ssh lab still
works before closing the first
```

Part H - When something does not work

What you see	Most likely cause	What to try
`ssh: Could not resolve hostname lab`	the configuration file is missing or in the wrong place, or it has a `.txt` extension	Check `%USERPROFILE%\.ssh\config` exists with no extension (`dir $env:USERPROFILE\.ssh`). Check the `Host` line spelling.
`Bad configuration option` / `Bad indentation`	a typo in the configuration file	Indent option lines with spaces; check each keyword's spelling.
`Connection refused`	the Pi is on, but no SSH server is listening (SSH not enabled), or it is still booting	Wait a minute. If it persists, enable SSH (`sudo raspi-config`, Interface Options) with a screen, or re-write the card with SSH enabled.
`Connection timed out`	different networks, VPN on, or a network that isolates devices	Same network; VPN off; hotspot; see Parts E and F.
`Permission denied (publickey)`	the Pi accepts only keys and yours is not installed, or the file permissions are wrong	Repeat Part B2 step 2; on the Pi: `chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys`.
Still asks for the password after step B2	the key was not appended, or a different key is offered	`ssh -v lab` shows which keys are tried; repeat step 2; check you did not overwrite `authorized_keys` with `>` instead of `>>`.
`REMOTE HOST IDENTIFICATION HAS CHANGED`	the Pi was re-written	`ssh-keygen -R raspi-lab.local` then connect again.
`scp: ... No such file or directory`	wrong path, or the destination folder does not exist	`ssh lab "ls ~"` to look; create folders with `ssh lab "mkdir -p ~/folder"`.
`scp` copies to a file called `lab` on your laptop	the colon is missing (`scp test.txt lab`)	Write `lab:~/`, with the colon.
VNC: cannot connect	VNC not enabled, wrong address, Pi not on the same network, or the viewer and server do not match	Enable VNC (D2); check the address with `hostname -I`; see the Wayland/X11 note in D2.
VNC: black or tiny desktop	no monitor attached, so no resolution	Set a resolution in `raspi-config` -> Display Options.
The Pi does not join the second Wi-Fi	typo; wrong country; the network is not 2.4 or 5 GHz compatible; enterprise network	Check `nmcli connection show` and `nmcli device wifi list`; correct the profile; use a hotspot to test.
`nmcli: command not found`	an older OS release without NetworkManager	See the older-release note in E2.

Check yourself

1. Why is `ssh lab` shorter than `ssh piuser@raspi-lab.local`? Where does the extra information come from?
2. What is the difference between a private key and a public key? Which one may you share?
3. You have two laptops. How do you give both of them key access to one Pi?
4. What does tmux do for you, and which keys detach a session?
5. In `scp a.txt lab:~/`, which part is the source and which the destination? How do you copy from the Pi to the current folder?
6. Why do we not put a 500 MB model in a git repository? What do we use instead?
7. When is SSH enough, and when do you want VNC?
8. A Pi has no monitor. You connect with VNC and see a tiny or black desktop. What do you change?
9. How do you add a Wi-Fi network to a Pi that is not currently in range of it?
10. `raspi-lab.local` does not resolve on a guest Wi-Fi. Name two ways to find the Pi's address.
11. Name two safe ways and one unsafe way to reach a Pi in another network.
12. Name three security habits for a Pi.

> **NOTE Answers**
>
> 1) The configuration file maps the nickname to the host name and the user. 2) The private key is secret and stays on the laptop; the public key is placed on the Pi and may be shared. 3) Create a key pair on each laptop and append both public keys to the Pi's `authorized_keys`. 4) tmux keeps a terminal session running on the Pi when you disconnect; press Ctrl+b then d. 5) `a.txt` is the source, `lab:~/` the destination (the Pi's home folder); `scp lab:file .`. 6) Large files do not suit git; use scp or an SFTP program (and a .gitignore). 7) SSH for commands, installs and scripts; VNC for graphical programs and previews. 8) Set a resolution in raspi-config, Display Options. 9) Create a saved profile with `nmcli connection add type wifi ...`. 10) The router's device list; `arp -a`/`ping` from the laptop; a monitor with `hostname -I`. 11) Safe: a private-network service, an approved VPN, carrying the Pi; unsafe: forwarding SSH on the router. 12) Strong password, updates, keys, no exposure to the Internet, backups.

Command cheat sheet

Task	Command
Connect with a nickname	`ssh lab`
Create a key pair	`ssh-keygen -t ed25519`
Copy a file to the Pi	`scp file.txt lab:~/`
Copy a file from the Pi	`scp lab:file.txt .`
Copy a folder	`scp -r folder lab:~/`
Run one command on the Pi	`ssh lab "hostname -I"`
Start / leave / resume tmux	`tmux new -s work` / Ctrl+b d / `tmux attach -t work`
Enable VNC	`sudo raspi-config` -> Interface Options -> VNC
List Wi-Fi in range	`nmcli device wifi list`
Saved connections	`nmcli connection show`
Add a Wi-Fi profile	`sudo nmcli connection add type wifi con-name NAME ifname wlan0 ssid "SSID" wifi-sec.key-mgmt wpa-psk wifi-sec.psk "PASSWORD"`
Pi's IP address	`hostname -I`
Update the system	`sudo apt-get update && sudo apt upgrade`
Shut down	`sudo shutdown -h now`
Sync code	`git pull` / `git add . git commit -m "..." git push`

Instructor notes

TEACHING TIP **A 60-minute lab**

- **10 min:** the rough edges (A1); demonstrate `ssh lab` versus the long form. **15 min:** students create the configuration file and a key (B1, B2). **15 min:** scp both ways and an SFTP program (C1, C2). **15 min:** VNC (D). **5 min:** where networks break (E, F).
- **Typical mistakes:** the config file saved as `config.txt`; the colon missing in scp; the private key copied instead of the public key; `>` instead of `>>` when adding a key (overwrites other keys); VNC with no resolution set.
- **Mini-assessment:** each student shows `ssh lab` working without a password and a file copied both ways (a screenshot of `ls -l` and the scp lines).
- **Policy:** warn students not to install remote-access software on school or work computers without permission.

Glossary

Term	Meaning
SSH configuration file	A text file (`~/.ssh/config`) that gives nicknames and options to SSH destinations.
Key pair	A private key (secret) and a public key (shareable) used to prove identity without a password.
`authorized_keys`	The file on the Pi listing the public keys allowed to log in.
tmux	A terminal multiplexer: keeps sessions alive on the Pi when you disconnect.
scp, SFTP	Copy over SSH from the command line; a file-transfer protocol with two-pane programs.
git, repository	A version-control system; the shared online store of a project.
VNC	Virtual Network Computing: remote control of a graphical desktop.
NetworkManager, nmcli	The network service of current Raspberry Pi OS and its command-line tool.
DHCP reservation	A router setting that always gives one device the same IP address.
NAT	The router technique that hides private addresses behind one public address.
Link-local address	A `169.254.x.x` address a device gives itself when no DHCP server answers.
VPN, overlay network	A private network built over the Internet that makes distant devices behave as if they were together.
Port forwarding	A router rule that exposes a device's port to the Internet (not recommended for SSH).

Appendix A - My Pis and my laptops

One row per Pi or laptop. (Do not write passwords here.)

	Pi 1	Pi 2	Laptop 1	Laptop 2
Board / RAM				
Host name				
SSH nickname				
User name				
Where it lives				
IP address (can change)				
Wi-Fi networks saved				
SSH key added? (yes/no)				
VNC enabled? (yes/no)				
Notes				

Appendix B - My notes and questions

Acknowledgments and references

This course is a compilation: it combines material from the sources below with new explanations, examples, diagrams, exercises and comments written for this course. The original authors are gratefully acknowledged.

[1] Rovai, M. (2025). *Edge AI Engineering: Hands-on with the Raspberry Pi*. Open textbook for the course IESTI05, Federal University of Itajubá (UNIFEI), Brazil. Version of 23 October 2025. Online edition: https://mjrovai.github.io/EdgeML_Made_Ease_ebook/

[2] Rovai, M. (2026). *IESTI05 - Edge AI: Edge Machine Learning Systems Engineering*. Course slides, notebooks and video lectures (second semester 2026), UNIFEI, Brazil. Course videos listed by the author: Class 1 (course introduction) https://youtu.be/QAb3MFHwe78 ; Class 2 (Introduction to Linux) https://youtu.be/QkIHILtl0h4 ; Class 3 (Raspberry Pi setup walkthrough) https://youtu.be/YtjsM36LzVE (lectures in Portuguese).

[3] Rovai, M. (2026). *IESTI01 - TinyML: Machine Learning for Embedding Devices*. Course materials (first semester 2026), UNIFEI, Brazil. https://github.com/Mjrovai/UNIFEI-IESTI01-TinyML

[4] Rovai, M. *TinyML4D*: lectures, books, papers and tutorials for embedded machine-learning education. https://github.com/Mjrovai/TinyML4D

[5] Raspberry Pi Ltd. *Raspberry Pi documentation* (operating system, remote access, Raspberry Pi Connect, hardware). https://www.raspberrypi.com/documentation/

[6] OpenSSH project. *OpenSSH*: the SSH client and server used by Windows, macOS, Linux and Raspberry Pi OS (manual pages ssh, ssh_config, scp, ssh-keygen). https://www.openssh.com/

[7] FileZilla Project. *FileZilla Client*, a free SFTP program. https://filezilla-project.org/

[8] RealVNC. *VNC Viewer* and VNC Server. https://www.realvnc.com/

[9] Tailscale Inc. *Tailscale* documentation (private network between your own devices). https://tailscale.com/

The screenshots of the scp transfer, FileZilla, raspi-config and the VNC viewer are adapted from reference [1]. All other figures and the animation were created for this course. The animation voice-over is synthetic speech.